

C++ Course

Materials for Students

Paweł Kisielewicz pkisiel@gmail.com

Krakow, 20 June 2026

Contents

1	Introduction	5
2	English Version Plan	6
2.1	Goal	6
2.2	Learning Path	6
2.3	Translation Batches	6
2.3.1	Batch 1	6
2.3.2	Batch 2	6
2.3.3	Batch 3	6
2.3.4	Batch 4	7
2.3.5	Batch 5	7
2.3.6	Batch 6	7
2.4	Editorial Rules	7
2.5	YouTube Support	7
2.6	Current Downloadable English Edition	7
3	00 - Start	9
3.1	Learning Goals	9
3.2	What You Need	9
3.3	First Compiler Check	9
3.4	First Program	9
3.5	Editor	10
3.6	Git Basics	10
3.7	CMake in One Sentence	10
3.8	Student Checklist	10
3.9	Next Step	10
4	01 - C++ Basics	11
4.1	Learning Goals	11
4.2	Minimal Program Structure	11
4.3	Output	11
4.4	Input	12
4.5	Basic Types	12
4.6	Simple Calculation	12
4.7	Formatting Output	13
4.8	Common Mistakes	13
4.9	Practice Tasks	13
4.10	Student Checklist	13
4.11	Next Step	14
5	02 - Control Flow and Loops	15
5.1	Learning Goals	15
5.2	Conditions	15
5.3	Multiple Conditions	16
5.4	<code>switch</code>	16
5.5	<code>for</code> Loop	17

5.6	<code>while</code> Loop	17
5.7	<code>do while</code> Loop	17
5.8	<code>break</code> and <code>continue</code>	18
5.9	Common Mistakes	18
5.10	Bridge Exercise: Minimum, Maximum, and Average	18
	5.10.1 Requirements	18
	5.10.2 Suggested Work Plan	19
	5.10.3 Check Scenarios	19
5.11	Practice Tasks	19
5.12	Student Checklist	19
6	03 - Functions, Arrays, and Strings	20
6.1	Learning Goals	20
6.2	Why Functions Matter	20
6.3	Declaration and Definition	20
6.4	Parameters and Return Values	21
6.5	Local Variables and Scope	21
6.6	One-Dimensional Arrays	21
6.7	Array Minimum and Maximum	21
6.8	Two-Dimensional Arrays	22
6.9	Strings	22
6.10	Reading a Full Line	22
6.11	Bridge Exercise: GADERYPOLUKI Cipher	23
6.12	Required Function	23
6.13	Suggested Work Plan	23
6.14	Check Scenarios	23
6.15	Extra Tasks	24
6.16	Common Mistakes	24
6.17	Practice Tasks	24
6.18	Student Checklist	24
7	04 - Pointers, References, and Memory	25
7.1	Learning Goals	25
7.2	Value and Address	25
7.3	Pointers	25
7.4	<code>nullptr</code>	26
7.5	References	26
7.6	Pointer or Reference	26
7.7	Passing to Functions	26
7.8	<code>const</code> References	27
7.9	Arrays and Pointers	27
7.10	Memory Safety Rules	27
7.11	Practice Tasks	28
7.12	Checklist	28
8	05 - Files and Exceptions	29
8.1	Learning Goals	29
8.2	Writing a Text File	29
8.3	Appending Data	29
8.4	Reading a Text File	30
8.5	Reading Numbers with Validation	30
8.6	Simple Configuration Format	31
8.7	Exceptions	31
8.8	Error-Handling Strategy	31
8.9	Bridge Exercise: File Menu	31
8.10	Practice Tasks	32
8.11	Checklist	32
9	06 - Object-Oriented Programming	33

9.1	Learning Goals	33
9.2	Classes and Objects	33
9.3	Encapsulation	34
9.4	Constructors	34
9.5	<code>const</code> Methods and <code>this</code>	34
9.6	Inheritance	35
9.7	Virtual Methods and Polymorphism	35
9.8	Why the Virtual Destructor Matters	36
9.9	Operator Overloading	36
9.10	Bridge Exercise: Shapes and Polymorphism	37
9.11	Checklist	37
10	07 - STL and Data Structures	38
10.1	Learning Goals	38
10.2	Why STL Matters	38
10.3	<code>std::vector</code>	38
10.4	Vector of Records	39
10.5	Sorting	39
10.6	Searching with <code>std::find_if</code>	40
10.7	<code>std::map</code>	40
10.8	Stack, Queue, and List	41
10.9	Choosing a Container	41
10.10	Mini Project: HRMS	41
10.11	Practice Tasks	42
10.12	Checklist	42
11	08 - Project Structure, Build, and Tests	43
11.1	Learning Goals	43
11.2	From One File to Several Files	43
11.3	Headers and Source Files	43
11.4	Include Guards	44
11.5	Compiling Multiple Files	44
11.6	Build Script	44
11.7	Syntax Checks	45
11.8	Simple Tests Without a Framework	45
11.9	Basic CI/CD Idea	45
11.10	Bridge Exercise: RPN Calculator	46
11.11	Practice Tasks	47
11.12	Checklist	47
12	09 - Modern C++	48
12.1	Learning Goals	48
12.2	<code>auto</code>	48
12.3	<code>nullptr</code>	48
12.4	Brace Initialization	49
12.5	Range-Based <code>for</code>	49
12.6	<code>enum class</code>	49
12.7	Type Aliases	49
12.8	<code>override</code>	49
12.9	Lambdas	50
12.10	Basic Move Semantics Idea	50
12.11	Namespaces	50
12.12	Classic Files and C++20 Modules	51
12.13	Basic Threads	51
12.14	Practice Tasks	51
12.15	Checklist	51
13	10 - Projects	52
13.1	Learning Goals	52

13.2 Project Categories	52
13.2.1 Console Tools	52
13.2.2 User-Facing Systems	52
13.2.3 Games	52
13.3 How to Choose a Project	53
13.4 Minimum Version First	53
13.5 Suggested Project Structure	53
13.6 README Checklist	53
13.7 Final Project Checklist	54
13.8 Grading Rubric	55
13.9 Technical Review Questions	55
13.10Suggested Project Topics	55
13.11Final Advice	56

Chapter 1

Introduction

- Author: Paweł Kisielewicz <pkisiel@gmail.com>
- Place and date: Krakow, 20 June 2026
- Version: 0.7.1-en

This ebook is the first complete English edition based on the Polish C++ learning materials. The Polish version remains the source of truth for now, while the English version covers the full learning path from setup to final projects.

The full course is organized from environment setup and C++ basics to functions, arrays, strings, files, exceptions, object-oriented programming, STL, build systems, testing, and final projects.

Chapter 2

English Version Plan

2.1 Goal

The English edition should become a downloadable version of the C++ course for students who want to learn from the same material as the Polish course.

The current English edition covers the full learning path from **00-start** to **10-projects**. Further work should focus on review, examples, consistency, and future polishing rather than creating the first structure.

2.2 Learning Path

1. **00-start** - development environment, compiler, editor, Git, and CMake.
2. **01-basics** - first program, input/output, types, variables, and operators.
3. **02-control-flow-loops** - conditions, **switch**, loops, **break**, **continue**.
4. **03-functions-arrays-strings** - functions, arrays, matrices, and strings.
5. **04-pointers-references-memory** - addresses, pointers, references, and safe memory habits.
6. **05-files-exceptions** - text files, validation, and exceptions.
7. **06-oop** - classes, objects, constructors, encapsulation, inheritance, and operators.
8. **07-stl-data-structures** - STL containers, algorithms, maps, stacks, queues, and lists.
9. **08-project-build-tests** - multi-file projects, build scripts, tests, and CI.
10. **09-modern-cpp** - selected modern C++ features, lambdas, move semantics, namespaces, and threads.
11. **10-projects** - final project topics, grading checklists, and project planning.

2.3 Translation Batches

2.3.1 Batch 1

- Main repository introduction - draft added.
- Ebook front matter - draft added.
- **00-start** - first English chapter added.
- **01-basics** - first English chapter added.

2.3.2 Batch 2

- **02-control-flow-loops** - first English chapter added.
- Bridge exercise: minimum, maximum, and average - included in the chapter.

2.3.3 Batch 3

- **03-functions-arrays-strings** - first English chapter added.
- Bridge exercise: GADERYPOLUKI cipher - included in the chapter.

2.3.4 Batch 4

- 04-pointers-references-memory - first English chapter added.
- 05-files-exceptions - first English chapter added.
- 06-oop - first English chapter added.
- Bridge exercise: figures and polymorphism - included in the chapter.

2.3.5 Batch 5

- 07-stl-data-structures - first English chapter added.
- 08-project-build-tests - first English chapter added.
- Bridge exercise: RPN calculator - included in the chapter.

2.3.6 Batch 6

- 09-modern-cpp - first English chapter added.
- 10-projects - first English chapter added.
- Project checklist and grading card - included in the project chapter.

2.4 Editorial Rules

- Keep the English text simple and practical.
- Do not translate code identifiers unless the example itself is rewritten.
- Prefer short paragraphs and clear section titles.
- Keep exercises close to the Polish version, but adjust wording for an English-speaking student.
- Do not include archive materials in the ebook.
- Keep the Polish and English ebook versions buildable by the same script.

2.5 YouTube Support

The first bilingual video format should be short quizzes:

- Polish question first.
- English question second.
- One C++ concept per video.
- Link to the repository and both ebook downloads in the description.

Example:

C++ Quiz #1

What will this program print?

```
int x = 5;
std::cout << x++ << " " << x;
```

- A) 5 5
- B) 5 6
- C) 6 6

Answer: B) 5 6, because postfix `x++` returns the current value first and increments the variable afterwards.

2.6 Current Downloadable English Edition

The current downloadable English edition is:

- title: C++ Course
- version: 0.7.1-en
- scope: full learning path 00-10
- output files:

- `ebook/download/cpp-course-en.pdf`
- `ebook/download/cpp-course-en.epub`

Chapter 3

00 - Start

This chapter prepares the technical environment for learning C++. It is meant for students who are starting with a compiler, code editor, terminal, Git, and simple project structure.

3.1 Learning Goals

After this chapter you should be able to:

- explain what a C++ compiler does,
- compile and run a simple `.cpp` file,
- use a terminal for basic commands,
- open and edit source files in Visual Studio Code or another editor,
- understand why Git is useful during programming exercises,
- understand the role of `CMakeLists.txt` in larger projects,
- move safely to the first C++ programming chapter.

3.2 What You Need

You need:

- a C++ compiler with C++17 support,
- a text editor or IDE,
- a terminal,
- Git,
- access to this repository.

On macOS and Linux, the compiler is often available as `c++`, `clang++`, or `g++`. On Windows, students usually use MSYS2, Visual Studio Build Tools, or an IDE configured by the teacher.

3.3 First Compiler Check

Open a terminal and run:

```
c++ --version
```

If the command prints compiler information, you can continue. If it says that the command is missing, install a compiler before moving forward.

3.4 First Program

Create a file named `hello.cpp`:

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello, C++!" << std::endl;  
    return 0;  
}
```

Compile it:

```
c++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello
```

Run it:

```
./hello
```

Expected output:

```
Hello, C++!
```

3.5 Editor

Use an editor that lets you:

- open folders,
- edit `.cpp` files,
- open an integrated terminal,
- see compiler errors clearly.

Visual Studio Code is a practical default for beginners, but the course does not depend on one specific editor.

3.6 Git Basics

Git helps you save progress and review changes. For this course, the most important commands are:

```
git status  
git add file.cpp  
git commit -m "Describe the change"
```

At the beginning, treat Git as a history of your learning steps. Commit small, working changes.

3.7 CMake in One Sentence

CMake is a build configuration tool. In small exercises you can compile one file manually. In larger projects, CMake or a build script describes how many source files should be compiled together.

3.8 Student Checklist

Before going to the next chapter, make sure you can:

- open a terminal,
- check the compiler version,
- compile `hello.cpp`,
- run the generated program,
- explain what source file and executable mean,
- check Git status in a repository.

3.9 Next Step

Go to chapter 01 - Basics and write your first simple console programs.

Chapter 4

01 - C++ Basics

This chapter is the first programming block of the course. It introduces the structure of a C++ program, input and output, basic data types, variables, operators, and simple console tasks.

4.1 Learning Goals

After this chapter you should be able to:

- describe the role of an algorithm, a program, and the `main` function,
- write, compile, and run a simple C++ program,
- use `std::cout`, `std::cin`, and `std::getline`,
- choose basic data types for a simple problem,
- declare and initialize variables,
- perform simple calculations,
- format basic text output,
- solve small console exercises.

4.2 Minimal Program Structure

Most beginner programs in this course start like this:

```
#include <iostream>

int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

Important parts:

- `#include <iostream>` makes console input/output available,
- `int main()` is the starting point of the program,
- statements inside `{ }` are executed in order,
- `return 0;` means the program finished successfully.

4.3 Output

Use `std::cout` to print text:

```
#include <iostream>

int main() {
    std::cout << "C++ basics" << std::endl;
```

```
    std::cout << "Second line" << std::endl;
}
```

`std::endl` ends the current line.

4.4 Input

Use `std::cin` to read a value:

```
#include <iostream>
#include <string>

int main() {
    std::string name;

    std::cout << "Your name: ";
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;
}
```

Use `std::getline` when you want to read a full line with spaces:

```
#include <iostream>
#include <string>

int main() {
    std::string fullName;

    std::cout << "Full name: ";
    std::getline(std::cin, fullName);

    std::cout << "Hello, " << fullName << "!" << std::endl;
}
```

4.5 Basic Types

Common beginner types:

Type	Example use
<code>int</code>	whole numbers
<code>double</code>	decimal numbers
<code>char</code>	one character
<code>bool</code>	true or false
<code>std::string</code>	text

Example:

```
int age = 20;
double price = 19.99;
bool active = true;
std::string city = "Krakow";
```

4.6 Simple Calculation

```
#include <iostream>

int main() {
```

```

double a = 0.0;
double b = 0.0;

std::cout << "First number: ";
std::cin >> a;

std::cout << "Second number: ";
std::cin >> b;

std::cout << "Sum: " << a + b << std::endl;
std::cout << "Product: " << a * b << std::endl;
}

```

4.7 Formatting Output

For decimal precision, use `<iomanip>`:

```

#include <iomanip>
#include <iostream>

int main() {
    double value = 10.0 / 3.0;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << value << std::endl;
}

```

Expected output:

3.33

4.8 Common Mistakes

- forgetting `#include <iostream>`,
- missing semicolon at the end of a statement,
- using `int` when the result should contain decimals,
- mixing `std::cin >> value` and `std::getline` without understanding how input is buffered,
- ignoring compiler warnings.

4.9 Practice Tasks

1. Print your name, city, and favorite programming topic.
2. Read two integers and print their sum, difference, and product.
3. Read a rectangle width and height, then print its area.
4. Read a temperature in Celsius and print the value in Fahrenheit.
5. Read a full name with spaces and print a greeting.

4.10 Student Checklist

Before moving to control flow and loops, make sure you can:

- write a minimal C++ program,
- compile it with `-std=c++17 -Wall -Wextra -pedantic`,
- read one value from the user,
- read one full line from the user,
- choose between `int`, `double`, `bool`, and `std::string`,
- format a decimal number with two digits after the decimal point.

4.11 Next Step

The next topic is control flow: `if`, `else`, `switch`, and loops.

Chapter 5

02 - Control Flow and Loops

This chapter teaches how a program makes decisions and repeats instructions. These tools are necessary for almost every non-trivial console program.

5.1 Learning Goals

After this chapter you should be able to:

- use `if`, `else if`, and `else`,
- use comparison and logical operators,
- use `switch` for simple menu-like choices,
- use the conditional operator `?:` in simple expressions,
- write loops with `for`, `while`, and `do while`,
- choose the right loop type for a problem,
- use `break` and `continue` in simple cases,
- solve tasks that require decisions and repetition.

5.2 Conditions

Use `if` when the program should execute code only in some situations:

```
#include <iostream>

int main() {
    int age = 0;

    std::cout << "Age: ";
    std::cin >> age;

    if (age >= 18) {
        std::cout << "Adult" << std::endl;
    } else {
        std::cout << "Minor" << std::endl;
    }
}
```

Common comparison operators:

Operator	Meaning
<code>==</code>	equal
<code>!=</code>	not equal
<code><</code>	less than
<code><=</code>	less than or equal

Operator	Meaning
>	greater than
>=	greater than or equal

Logical operators:

Operator	Meaning
&&	and
	or
!	not

5.3 Multiple Conditions

Use `else if` when there are several possible ranges or categories:

```
#include <iostream>

int main() {
    int points = 0;

    std::cout << "Points: ";
    std::cin >> points;

    if (points >= 90) {
        std::cout << "Very good" << std::endl;
    } else if (points >= 60) {
        std::cout << "Passed" << std::endl;
    } else {
        std::cout << "Failed" << std::endl;
    }
}
```

5.4 switch

Use `switch` for simple choices based on one value:

```
#include <iostream>

int main() {
    int option = 0;

    std::cout << "1. Add\n";
    std::cout << "2. Remove\n";
    std::cout << "Option: ";
    std::cin >> option;

    switch (option) {
        case 1:
            std::cout << "Add selected" << std::endl;
            break;
        case 2:
            std::cout << "Remove selected" << std::endl;
            break;
        default:
            std::cout << "Unknown option" << std::endl;
    }
}
```

```

        break;
    }
}

```

Remember `break`. Without it, execution continues into the next case.

5.5 for Loop

Use `for` when you know how many times the loop should run:

```

#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::cout << i << std::endl;
    }
}

```

Typical uses:

- counting from 1 to `n`,
- summing numbers,
- printing a table,
- iterating over an index range.

5.6 while Loop

Use `while` when the loop depends on a condition that may change during input:

```

#include <iostream>

int main() {
    int number = 0;

    while (number != 5) {
        std::cout << "Enter 5 to stop: ";
        std::cin >> number;
    }
}

```

This loop may execute zero times if the condition is false at the beginning.

5.7 do while Loop

Use `do while` when the body should run at least once:

```

#include <iostream>

int main() {
    int option = 0;

    do {
        std::cout << "1. Continue\n";
        std::cout << "0. Exit\n";
        std::cout << "Option: ";
        std::cin >> option;
    } while (option != 0);
}

```

This is useful for simple console menus.

5.8 break and continue

`break` exits a loop:

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) {
        break;
    }
    std::cout << i << std::endl;
}
```

`continue` skips the rest of the current iteration:

```
for (int i = 1; i <= 10; ++i) {
    if (i % 2 == 0) {
        continue;
    }
    std::cout << i << std::endl;
}
```

5.9 Common Mistakes

- writing `=` instead of `==` in a condition,
- adding an accidental semicolon after `if` or `while`,
- forgetting `break` in `switch`,
- creating an infinite loop because the loop variable never changes,
- dividing by zero when no valid input was provided,
- using `for` for input that should end based on user command.

5.10 Bridge Exercise: Minimum, Maximum, and Average

Write a console program that reads real numbers from the user and calculates:

- minimum value,
- maximum value,
- arithmetic average,
- number of valid values.

The user should be able to finish input with a portable text command, for example `q`, `end`, or an empty line.

Example session:

```
Enter a number or q to finish: 10
Enter a number or q to finish: -2.5
Enter a number or q to finish: 4.5
Enter a number or q to finish: q
```

```
Count: 3
Minimum: -2.5
Maximum: 10
Average: 4
```

5.10.1 Requirements

The program should:

- use C++17,
- use a `while` or `do while` loop,
- store the sum in a `double`,
- count valid values,

- update minimum and maximum after each valid value,
- detect invalid input,
- avoid division by zero when no numbers were entered,
- compile with `-Wall -Wextra -pedantic`.

5.10.2 Suggested Work Plan

1. Write a loop that reads text until the user enters `q`.
2. Add a counter for valid numbers.
3. Add sum and average calculation.
4. Add minimum and maximum.
5. Add a message for the empty data case.
6. Add invalid input handling, for example for `abc`.

5.10.3 Check Scenarios

1. Enter 10, -2.5, 4.5, then `q`.
2. Enter only one number and check that minimum, maximum, and average are the same.
3. Finish without entering any number and check the message.
4. Enter invalid text such as `abc` and check that the program asks again.
5. Enter several negative numbers and verify minimum and maximum.

5.11 Practice Tasks

1. Read a number and print whether it is positive, negative, or zero.
2. Read a menu option and handle it with `switch`.
3. Print numbers from 1 to 100 with a `for` loop.
4. Sum numbers from 1 to `n`.
5. Read numbers until the user enters 0.
6. Print a multiplication table.
7. Skip even numbers using `continue`.
8. Stop searching when a target value is found using `break`.

5.12 Student Checklist

Before moving to functions, arrays, and strings, make sure you can:

- write an `if/else` condition,
- use comparison and logical operators,
- use a `switch` with `break`,
- choose between `for`, `while`, and `do while`,
- write a loop that depends on user input,
- explain when `break` and `continue` are useful,
- solve the minimum, maximum, and average bridge exercise.

Chapter 6

03 - Functions, Arrays, and Strings

This chapter teaches how to split a program into smaller parts and how to work with simple collections of data. You will use functions, one-dimensional arrays, two-dimensional arrays, and `std::string`.

6.1 Learning Goals

After this chapter you should be able to:

- declare and define your own functions,
- pass arguments to functions,
- return values from functions,
- understand the difference between a declaration and a definition,
- use one-dimensional arrays,
- use simple two-dimensional arrays,
- iterate over array elements,
- perform basic operations on `std::string`,
- split a larger task into smaller functions.

6.2 Why Functions Matter

A function gives a name to a piece of logic. Instead of writing everything in `main`, you can move a calculation into a separate function:

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3) << std::endl;
}
```

This makes code easier to read, test, and reuse.

6.3 Declaration and Definition

A declaration tells the compiler that a function exists:

```
int add(int a, int b);
```

A definition contains the function body:

```
int add(int a, int b) {
    return a + b;
}
```

In small programs, both can be in one file. In larger projects, declarations often go into header files and definitions go into `.cpp` files.

6.4 Parameters and Return Values

Parameters are input values for a function. `return` sends a result back:

```
double rectangleArea(double width, double height) {
    return width * height;
}
```

Use clear names. A function named `rectangleArea` is easier to understand than one named `calc`.

6.5 Local Variables and Scope

Variables declared inside a function are local to that function:

```
int square(int value) {
    int result = value * value;
    return result;
}
```

`result` cannot be used outside `square`. This is useful because it prevents unrelated parts of the program from accidentally changing each other.

6.6 One-Dimensional Arrays

An array stores several values of the same type:

```
#include <iostream>

int main() {
    int numbers[5] = {4, 2, 9, 1, 7};

    for (int i = 0; i < 5; ++i) {
        std::cout << numbers[i] << std::endl;
    }
}
```

Array indexes start at 0. For an array of size 5, valid indexes are 0, 1, 2, 3, and 4.

6.7 Array Minimum and Maximum

```
#include <iostream>

int main() {
    int numbers[5] = {4, 2, 9, 1, 7};
    int minimum = numbers[0];
    int maximum = numbers[0];

    for (int i = 1; i < 5; ++i) {
        if (numbers[i] < minimum) {
            minimum = numbers[i];
        }
        if (numbers[i] > maximum) {
```

```

        maximum = numbers[i];
    }
}

std::cout << "Minimum: " << minimum << std::endl;
std::cout << "Maximum: " << maximum << std::endl;
}

```

The first element is a practical initial value when the array is not empty.

6.8 Two-Dimensional Arrays

A two-dimensional array can represent a table or a small matrix:

```

#include <iostream>

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int row = 0; row < 2; ++row) {
        for (int col = 0; col < 3; ++col) {
            std::cout << matrix[row][col] << " ";
        }
        std::cout << std::endl;
    }
}

```

Nested loops are the usual way to process rows and columns.

6.9 Strings

`std::string` stores text:

```

#include <iostream>
#include <string>

int main() {
    std::string text = "C++";

    std::cout << text << std::endl;
    std::cout << "Length: " << text.size() << std::endl;
}

```

You can iterate over characters:

```

for (char ch : text) {
    std::cout << ch << std::endl;
}

```

6.10 Reading a Full Line

Use `std::getline` when the text may contain spaces:

```

#include <iostream>
#include <string>

int main() {

```

```

std::string sentence;

std::cout << "Enter text: ";
std::getline(std::cin, sentence);

std::cout << "You entered: " << sentence << std::endl;
}

```

6.11 Bridge Exercise: GADERYPOLUKI Cipher

Write a console program that encrypts text with the GADERYPOLUKI substitution cipher.

The cipher uses replacement pairs:

GA DE RY PO LU KI

Rules:

- G becomes A,
- A becomes G,
- D becomes E,
- E becomes D,
- the same rule applies to every pair,
- characters not present in the key stay unchanged.

Example:

PROGRAM -> OYPAYGM

6.12 Required Function

The program should contain a function:

```
std::string encryptGaderypoluki(const std::string& text);
```

The function should:

- take the input text as `std::string`,
- return a new encrypted `std::string`,
- not modify the input text directly,
- keep characters outside the key unchanged.

6.13 Suggested Work Plan

1. Write a helper function that replaces one character.
2. Test the pair G and A.
3. Add all remaining key pairs.
4. Write a function that loops over the whole string.
5. Read input with `std::getline`.
6. Test text containing spaces, digits, and punctuation.

6.14 Check Scenarios

1. Input PROGRAM; expected result: OYPAYGM.
2. Input GADERYPOLUKI; every letter should be replaced by its pair.
3. Input ALA 123; digits and spaces should stay unchanged.
4. Input an empty string; the program should not crash.

6.15 Extra Tasks

After the basic version works, add:

- lowercase support,
- preserving letter case,
- automatic decryption with the same function,
- tests for the encryption function,
- reading text from a file,
- writing encrypted text to a file,
- a custom substitution key.

6.16 Common Mistakes

- printing from the encryption function instead of returning a value,
- changing only one pair and forgetting the rest of the key,
- using `std::cin >> text` when the text may contain spaces,
- modifying the original string when a returned result would be clearer,
- mixing input/output code with encryption logic.

6.17 Practice Tasks

1. Write a function `isEven`.
2. Write a function `maxOfTwo`.
3. Calculate the average of an array.
4. Find the minimum and maximum in an array.
5. Print a two-dimensional array as a table.
6. Count letters in a string.
7. Check whether a string is a palindrome.
8. Implement the GADERYPOLUKI cipher.

6.18 Student Checklist

Before moving to pointers, references, and memory, make sure you can:

- write a function with parameters and a return value,
- explain local variable scope,
- iterate over an array by index,
- process a two-dimensional array with nested loops,
- read a full line with `std::getline`,
- iterate over characters in `std::string`,
- split a string-processing task into small functions.

Chapter 7

04 - Pointers, References, and Memory

This chapter introduces addresses, pointers, references, and basic memory safety habits. These topics are powerful, but they require discipline because small mistakes can produce hard-to-diagnose bugs.

7.1 Learning Goals

After this chapter you should be able to:

- distinguish a variable value from its address,
- use the address-of operator `&`,
- declare and initialize pointers,
- understand `nullptr`,
- dereference a pointer with `*`,
- use references,
- pass data to functions by value, pointer, and reference,
- recognize common memory safety problems.

7.2 Value and Address

A variable has a value and an address in memory:

```
#include <iostream>

int main() {
    int number = 42;

    std::cout << "Value: " << number << std::endl;
    std::cout << "Address: " << &number << std::endl;
}
```

`number` reads the value. `&number` reads the address.

7.3 Pointers

A pointer stores an address:

```
#include <iostream>

int main() {
    int number = 42;
    int* pointer = &number;
}
```

```

    std::cout << pointer << std::endl;
    std::cout << *pointer << std::endl;
}

```

`pointer` stores the address. `*pointer` reads the value stored at that address. This is called dereferencing.

7.4 nullptr

If a pointer does not point to a valid object, initialize it with `nullptr`:

```
int* pointer = nullptr;
```

Before dereferencing a pointer, check whether it is not empty:

```

if (pointer != nullptr) {
    std::cout << *pointer << std::endl;
}

```

This habit prevents many beginner-level pointer mistakes.

7.5 References

A reference is another name for an existing object:

```

#include <iostream>

int main() {
    int value = 10;
    int& ref = value;

    ref = 20;

    std::cout << value << std::endl;
}

```

The program prints 20 because `ref` refers to `value`.

7.6 Pointer or Reference

Use a reference when the argument must exist:

```

void increment(int& value) {
    ++value;
}

```

Use a pointer when the argument may be missing:

```

void incrementIfPresent(int* value) {
    if (value != nullptr) {
        ++(*value);
    }
}

```

This is a practical rule for beginner code. It makes the intent visible in the function signature.

7.7 Passing to Functions

Passing by value copies the argument:

```
void changeCopy(int value) {
    value = 100;
}
```

Passing by reference can modify the original value:

```
void changeOriginal(int& value) {
    value = 100;
}
```

Passing by pointer can also modify the original value, but the function should handle `nullptr`:

```
void changeIfPresent(int* value) {
    if (value != nullptr) {
        *value = 100;
    }
}
```

7.8 const References

Use `const` references when a function should read a larger object without copying it:

```
#include <iostream>
#include <string>

void printName(const std::string& name) {
    std::cout << name << std::endl;
}
```

The function cannot modify `name`, and it avoids copying the string.

7.9 Arrays and Pointers

An array name can be used as a pointer to the first element:

```
#include <iostream>

int main() {
    int numbers[3] = {10, 20, 30};
    int* p = numbers;

    for (int i = 0; i < 3; ++i) {
        std::cout << p[i] << std::endl;
    }
}
```

Valid indexes for this array are 0, 1, and 2. Accessing `p[3]` is outside the array and is an error.

7.10 Memory Safety Rules

Use these rules consistently:

- initialize pointers,
- prefer `nullptr` for an empty pointer,
- check a pointer before dereferencing it,
- do not return the address of a local variable,
- do not access an array outside its valid range,
- prefer references when a value is required,
- prefer `const` references for read-only larger objects,
- keep variables close to the place where they are used.

Do not write code like this:

```
int* invalid() {  
    int local = 10;  
    return &local;  
}
```

`local` stops existing when the function ends. The returned pointer does not point to a valid object.

7.11 Practice Tasks

1. Write `printIfPresent`, which takes `int*` and prints the value only when the pointer is not `nullptr`.
2. Write `incrementIfPresent`, which takes `int*` and increments the value only when the pointer is not `nullptr`.
3. Write `increment`, which takes `int&`, and compare it with the pointer version.
4. Write a function that reads an array through a pointer and a size.
5. Find one previous function where `const std::string&` would be better than passing a string by value.

7.12 Checklist

Before moving on, make sure that your code:

- initializes every pointer,
- checks optional pointers before dereferencing,
- does not return addresses of local variables,
- does not read past the end of an array,
- uses references when a value is required,
- compiles without warnings.

Chapter 8

05 - Files and Exceptions

This chapter shows how a program can store data outside memory and how it can react to error situations. You will work with text files, input validation, and basic exceptions.

8.1 Learning Goals

After this chapter you should be able to:

- open a text file for writing,
- append data to a text file,
- open a text file for reading,
- read a file line by line,
- check whether a file operation succeeded,
- validate simple input data,
- use `try`, `throw`, and `catch`,
- choose a simple error-handling strategy for a small program.

8.2 Writing a Text File

Use `std::ofstream` to write to a file:

```
#include <fstream>
#include <iostream>

int main() {
    std::ofstream file("notes.txt");

    if (!file) {
        std::cout << "Cannot open file for writing." << std::endl;
        return 1;
    }

    file << "First line" << std::endl;
    file << "Second line" << std::endl;
}
```

Opening a file this way replaces the previous content.

8.3 Appending Data

Use `std::ios::app` to append data at the end:

```

#include <fstream>

int main() {
    std::ofstream file("notes.txt", std::ios::app);
    file << "Another line" << std::endl;
}

```

This is useful for logs, notes, and simple history files.

8.4 Reading a Text File

Use `std::ifstream` and `std::getline`:

```

#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream file("notes.txt");

    if (!file) {
        std::cout << "Cannot open file for reading." << std::endl;
        return 1;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::cout << line << std::endl;
    }
}

```

Always check whether the file was opened correctly before reading from it.

8.5 Reading Numbers with Validation

Input from a file can be invalid. Keep parsing and validation separate from the rest of the program:

```

#include <iostream>
#include <sstream>
#include <string>

bool parseInt(const std::string& text, int& result) {
    std::istringstream input(text);
    input >> result;
    return input && input.eof();
}

int main() {
    int value = 0;

    if (parseInt("123", value)) {
        std::cout << value << std::endl;
    } else {
        std::cout << "Invalid number" << std::endl;
    }
}

```

This design is easy to test because `parseInt` does not depend on a real file.

8.6 Simple Configuration Format

A practical beginner exercise is a small **key=value** file:

```
name=Alice
level=3
active=true
```

Read the file line by line, split each line at **=**, and validate the result. Empty lines and malformed lines should be handled deliberately.

8.7 Exceptions

An exception reports an error that cannot be handled at the current place in the code:

```
#include <iostream>
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0) {
        throw std::runtime_error("division by zero");
    }

    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 0) << std::endl;
    } catch (const std::runtime_error& error) {
        std::cout << "Error: " << error.what() << std::endl;
    }
}
```

Use exceptions for situations where normal execution cannot continue in the current function. Do not use them as a replacement for simple **if** statements.

8.8 Error-Handling Strategy

For small student programs, use this order:

1. Check expected problems with **if**, for example missing input or an empty value.
2. Return a status value when a function can fail in a normal way.
3. Throw an exception when the current function cannot sensibly continue.
4. Catch exceptions near the program boundary, for example in **main**.

The goal is not to use advanced error handling. The goal is to make failures visible and predictable.

8.9 Bridge Exercise: File Menu

Write a console program with a simple menu:

1. Add note
2. Show notes
3. Clear notes
0. Exit

Requirements:

- store notes in a text file,
- append new notes instead of replacing the whole file,

- read and print all notes,
- handle a missing file,
- validate the menu choice,
- keep file operations in separate functions.

8.10 Practice Tasks

1. Write a program that saves three lines to a file.
2. Modify it so that new lines are appended instead of overwriting the file.
3. Write a program that reads a file line by line and counts non-empty lines.
4. Write a function that parses one integer from a string.
5. Write a function that throws an exception for division by zero.

8.11 Checklist

Before moving on, make sure that your code:

- checks whether files open successfully,
- separates parsing from file reading,
- handles invalid input,
- catches exceptions by `const` reference,
- keeps user-facing error messages clear,
- compiles without warnings.

Chapter 9

06 - Object-Oriented Programming

This chapter introduces classes, objects, constructors, encapsulation, inheritance, and polymorphism. The goal is not to make every program object-oriented. The goal is to understand when objects help organize data and behavior.

9.1 Learning Goals

After this chapter you should be able to:

- define a class,
- create objects,
- use private fields,
- write constructors,
- write `const` methods,
- understand `this`,
- explain basic inheritance,
- use virtual methods,
- use `override`,
- overload a simple operator,
- build a small polymorphic program.

9.2 Classes and Objects

A class defines a type. An object is a value of that type:

```
#include <iostream>
#include <string>

class Student {
public:
    Student(const std::string& name, int points)
        : name_(name), points_(points) {

        void print() const {
            std::cout << name_ << ": " << points_ << std::endl;
        }

private:
    std::string name_;
    int points_;
};
```

```
int main() {
    Student student("Alice", 42);
    student.print();
}
```

The public part is the interface. The private part is implementation detail.

9.3 Encapsulation

Encapsulation means that the class protects its own data. Instead of making fields public, provide methods that keep the object valid:

```
class Counter {
public:
    void increment() {
        ++value_;
    }

    int value() const {
        return value_;
    }

private:
    int value_ = 0;
};
```

Code outside the class cannot accidentally assign an invalid internal state.

9.4 Constructors

A constructor prepares an object for use:

```
class Rectangle {
public:
    Rectangle(double width, double height)
        : width_(width), height_(height) {

    double area() const {
        return width_ * height_;
    }

private:
    double width_;
    double height_;
};
```

Prefer initializing fields in the initializer list.

9.5 const Methods and this

A method marked with `const` promises not to modify the object:

```
double area() const {
    return width_ * height_;
}
```

Inside a method, `this` points to the current object. You rarely need to write `this` explicitly in beginner code, but it helps explain what method calls do.

9.6 Inheritance

Inheritance lets one class reuse or specialize another class:

```
#include <iostream>
#include <string>

class Person {
public:
    explicit Person(const std::string& name) : name_(name) {
    }

    void printName() const {
        std::cout << name_ << std::endl;
    }

private:
    std::string name_;
};

class Teacher : public Person {
public:
    Teacher(const std::string& name, const std::string& subject)
        : Person(name), subject_(subject) {
    }

private:
    std::string subject_;
};
```

Use inheritance carefully. If one class is not a real specialized version of another class, composition is often clearer.

9.7 Virtual Methods and Polymorphism

Polymorphism means that code can call the same method through a base interface, while the actual implementation depends on the object type:

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual std::string description() const = 0;
};

class Square : public Shape {
public:
    explicit Square(double side) : side_(side) {
    }

    double area() const override {
        return side_ * side_;
    }
};
```

```

    std::string description() const override {
        return "square";
    }

private:
    double side_;
};

class Circle : public Shape {
public:
    explicit Circle(double radius) : radius_(radius) {

        double area() const override {
            return 3.14159 * radius_ * radius_;
        }

        std::string description() const override {
            return "circle";
        }

private:
    double radius_;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Square>(4.0));
    shapes.push_back(std::make_unique<Circle>(2.0));

    for (const auto& shape : shapes) {
        std::cout << shape->description()
                    << " : " << shape->area() << std::endl;
    }
}

```

Shape is abstract because it has pure virtual methods. Square and Circle provide concrete implementations.

9.8 Why the Virtual Destructor Matters

When a derived object is deleted through a base pointer, the base class should have a virtual destructor:

```
virtual ~Shape() = default;
```

This is a standard safety rule for base classes used polymorphically.

9.9 Operator Overloading

Operator overloading lets a class define how an operator works for its objects:

```

class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {

        Point operator+(const Point& other) const {
            return Point(x_ + other.x_, y_ + other.y_);
        }
    }
}

```

```
private:
    int x_;
    int y_;
};
```

Use operator overloading only when the operator has a natural meaning for the type.

9.10 Bridge Exercise: Shapes and Polymorphism

Write a console program that:

- defines an abstract base class **Shape**,
- defines derived classes **Square** and **Circle**,
- calculates the area of each shape,
- returns a description of each shape,
- stores different shapes through base-class pointers,
- iterates over the collection and calls methods polymorphically.

Minimum technical requirements:

- C++17,
- pure virtual methods,
- virtual destructor in the base class,
- private fields in derived classes,
- **const** methods for read-only operations,
- **override** in derived classes,
- **std::vector** for the collection,
- **std::unique_ptr<Shape>** for ownership,
- compilation with **-Wall -Wextra -pedantic**.

Suggested steps:

1. Write the abstract class **Shape**.
2. Add **Square** and test area calculation.
3. Add **Circle** and a constant for pi.
4. Add **description** to every derived class.
5. Store shapes in one collection through base-class pointers.
6. Print every description and area in a loop.
7. Add validation for dimensions.

Extra tasks:

- add **Rectangle**,
- add **Triangle**,
- add perimeter calculation,
- sort shapes by area,
- calculate total area,
- save the shape list to a file.

9.11 Checklist

Before moving on, make sure that your code:

- keeps fields private,
- initializes objects through constructors,
- marks read-only methods as **const**,
- uses **override** for overridden virtual methods,
- uses a virtual destructor in polymorphic base classes,
- separates object logic from input/output when possible,
- compiles without warnings.

Chapter 10

07 - STL and Data Structures

This chapter introduces the Standard Template Library containers and a few common algorithms. You will learn how to choose a container, store records, search data, sort values, and combine containers in a small program.

10.1 Learning Goals

After this chapter you should be able to:

- choose a container for a simple task,
- use `std::vector` for lists of values,
- iterate over containers with a range-based `for` loop,
- add, remove, and search elements,
- use `std::map` for key-value data,
- use `std::stack` and `std::queue`,
- use basic STL algorithms such as `std::sort` and `std::find_if`,
- combine containers with classes or structs.

10.2 Why STL Matters

The STL gives you tested building blocks. Instead of writing your own dynamic array, map, stack, or queue, use the standard container that matches the task.

For beginner projects, this usually means:

- `std::vector` for a list,
- `std::map` for lookup by key,
- `std::queue` for first-in-first-out processing,
- `std::stack` for last-in-first-out processing,
- algorithms from `<algorithm>` for searching, sorting, and filtering.

10.3 `std::vector`

`std::vector` stores a sequence of values:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
}
```

```

    numbers.push_back(30);

    for (int value : numbers) {
        std::cout << value << std::endl;
    }
}

```

Use `push_back` to add an element at the end. Use `size` to check how many elements are stored.

10.4 Vector of Records

Containers become more useful when they store structured data:

```

#include <iostream>
#include <string>
#include <vector>

struct Student {
    std::string name;
    int points;
};

int main() {
    std::vector<Student> students = {
        {"Alice", 42},
        {"Bob", 35},
        {"Carol", 50}
    };

    for (const Student& student : students) {
        std::cout << student.name << ": "
                  << student.points << std::endl;
    }
}

```

Use `const Student&` in the loop when you only read the element and do not want to copy it.

10.5 Sorting

Use `std::sort` from `<algorithm>`:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers = {5, 2, 9, 1};

    std::sort(numbers.begin(), numbers.end());

    for (int value : numbers) {
        std::cout << value << std::endl;
    }
}

```

For records, pass a comparison function or a lambda:

```

std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {

```



```
        return a.points > b.points;
    });
```

This sorts students by points from highest to lowest.

10.6 Searching with `std::find_if`

Use `std::find_if` when the search condition is more complex than equality:

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Student {
    std::string name;
    int points;
};

int main() {
    std::vector<Student> students = {
        {"Alice", 42},
        {"Bob", 35}
    };

    auto found = std::find_if(students.begin(), students.end(),
        [](const Student& student) {
            return student.name == "Bob";
        });

    if (found != students.end()) {
        std::cout << found->points << std::endl;
    }
}
```

The result is an iterator. Compare it with `end()` before using it.

10.7 `std::map`

`std::map` stores key-value pairs:

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> points;

    points["Alice"] = 42;
    points["Bob"] = 35;

    std::cout << points["Alice"] << std::endl;
}
```

Use `find` when you want to check whether a key exists:

```
auto it = points.find("Carol");
if (it != points.end()) {
    std::cout << it->second << std::endl;
}
```

This avoids accidentally creating a new entry.

10.8 Stack, Queue, and List

Use `std::stack` when the last value added should be processed first:

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> values;
    values.push(1);
    values.push(2);

    std::cout << values.top() << std::endl;
    values.pop();
}
```

Use `std::queue` when the first value added should be processed first:

```
#include <iostream>
#include <queue>

int main() {
    std::queue<std::string> tasks;
    tasks.push("compile");
    tasks.push("test");

    std::cout << tasks.front() << std::endl;
    tasks.pop();
}
```

Use `std::list` only when you really need frequent insertions and removals in the middle. For many beginner tasks, `std::vector` is the better default.

10.9 Choosing a Container

Use these practical rules:

- choose `std::vector` for ordered data and simple iteration,
- choose `std::map` when lookup by key is important,
- choose `std::queue` for tasks processed in arrival order,
- choose `std::stack` for reverse-order processing,
- choose `std::list` only when its insertion and removal behavior matters.

Start with the simplest container that matches the problem.

10.10 Mini Project: HRMS

Build a small human resources management program.

Minimum version:

- store employees in `std::vector`,
- each employee has an id, name, department, and salary,
- print all employees,
- find an employee by id,
- sort employees by salary,
- group or count employees by department with `std::map`.

Suggested structure:

```
struct Employee {  
    int id;  
    std::string name;  
    std::string department;  
    double salary;  
};
```

Keep operations in separate functions. This prepares the code for the next chapter, where projects are split into multiple files and tested.

10.11 Practice Tasks

1. Store five numbers in a vector and print them in sorted order.
2. Store students in a vector and find the student with the highest score.
3. Build a `std::map<std::string, int>` that counts words.
4. Simulate a queue of tasks with `std::queue`.
5. Use `std::stack` to reverse a sequence of values.
6. Extend the HRMS mini project with a department filter.

10.12 Checklist

Before moving on, make sure that your code:

- uses standard containers instead of manual arrays when appropriate,
- checks iterators before dereferencing,
- uses `const` references for read-only records,
- keeps container choice simple and justified,
- separates data operations into functions,
- compiles without warnings.

Chapter 11

08 - Project Structure, Build, and Tests

This chapter shows how to move from one `.cpp` file to a small C++ project with headers, source files, build commands, syntax checks, tests, and basic automation.

11.1 Learning Goals

After this chapter you should be able to:

- split a program into header and source files,
- explain the role of the preprocessor and `#include`,
- protect headers against multiple inclusion,
- create a minimal project structure,
- compile a project made of several files,
- check syntax without building a full program,
- write simple tests without an external framework,
- explain the basic idea of CI/CD.

11.2 From One File to Several Files

A one-file program is fine at the beginning. As the code grows, split it into:

- headers with declarations,
- source files with definitions,
- tests that check selected functions,
- a build directory for generated files.

A small structure can look like this:

```
calculator/  
  include/calculator.h  
  src/calculator.cpp  
  src/main.cpp  
  tests/calculator_tests.cpp  
  build/  
  build.sh
```

The exact layout can vary. The important rule is that source code and generated files are not mixed.

11.3 Headers and Source Files

A header declares what exists:

```
// include/calculator.h
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);

#endif
```

A source file defines how it works:

```
// src/calculator.cpp
#include "calculator.h"

int add(int a, int b) {
    return a + b;
}
```

main.cpp can use the function through the header:

```
#include <iostream>
#include "calculator.h"

int main() {
    std::cout << add(2, 3) << std::endl;
}
```

11.4 Include Guards

Include guards prevent the same header from being included more than once in a single translation unit:

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

// declarations

#endif
```

You may also see `#pragma once` in real projects. Include guards are explicit and portable, so they are a good teaching default.

11.5 Compiling Multiple Files

A simple manual compile command can look like this:

```
c++ -std=c++17 -Wall -Wextra -pedantic \
    -Iinclude \
    src/main.cpp src/calculator.cpp \
    -o build/calculator
```

Important parts:

- `-std=c++17` selects the language version,
- `-Wall -Wextra -pedantic` enables useful warnings,
- `-Iinclude` tells the compiler where to find project headers,
- several `.cpp` files are compiled and linked into one program.

11.6 Build Script

A small `build.sh` keeps commands repeatable:

```
#!/bin/sh
set -eu
```

```
mkdir -p build
```

```
c++ -std=c++17 -Wall -Wextra -pedantic \  
    -Iinclude \  
    src/main.cpp src/calculator.cpp \  
    -o build/calculator
```

```
c++ -std=c++17 -Wall -Wextra -pedantic \  
    -Iinclude \  
    tests/calculator_tests.cpp src/calculator.cpp \  
    -o build/calculator_tests
```

```
./build/calculator_tests
```

Generated files should stay in build/ and should not be committed.

11.7 Syntax Checks

Use `-fsyntax-only` when you want the compiler to check code without producing an executable:

```
c++ -std=c++17 -Wall -Wextra -pedantic -fsyntax-only src/main.cpp
```

This is useful for quick feedback and for checking starter files.

11.8 Simple Tests Without a Framework

A minimal test program can return 0 for success and non-zero for failure:

```
#include "calculator.h"
```

```
int main() {  
    if (add(2, 3) != 5) {  
        return 1;  
    }  
  
    if (add(-2, 2) != 0) {  
        return 1;  
    }  
  
    return 0;  
}
```

The build script can run the test executable. If the test returns 1, the script fails.

11.9 Basic CI/CD Idea

CI means that checks are run automatically after a change is pushed to the repository. For this course, a useful first CI setup would:

- install a compiler,
- compile examples,
- run simple tests,
- fail when a command returns an error.

CD is about automatic delivery or deployment. For this repository, CI is the more important concept for students.

11.10 Bridge Exercise: RPN Calculator

Build a small multi-file project that calculates arithmetic expressions by converting them to Reverse Polish Notation.

The program should:

- read one text expression,
- support real numbers,
- support +, -, *, /,
- respect multiplication and division precedence,
- support parentheses,
- convert the expression to Reverse Polish Notation,
- evaluate the result with a stack,
- report clear errors for invalid expressions.

Examples:

```
2 + 2 * 2 = 6
(2 + 2) * 2 = 8
10 / 2 + 3 = 8
```

Suggested structure:

```
rpn-calculator/
  include/rpn/
    token.h
    tokenizer.h
    rpn.h
    evaluator.h
  src/
    tokenizer.cpp
    rpn.cpp
    evaluator.cpp
    main.cpp
  tests/
    rpn_tests.cpp
  build/
    .gitignore
    build.sh
    README.md
```

Suggested functions:

```
std::vector<Token> tokenize(const std::string& expression);
std::vector<Token> toReversePolishNotation(const std::vector<Token>& tokens);
double evaluateReversePolishNotation(const std::vector<Token>& tokens);
```

Minimum requirements:

- C++17,
- separate tokenization from evaluation,
- use `std::vector<Token>`,
- use `std::vector` or `std::stack` as the evaluation stack,
- handle division by zero,
- detect invalid parentheses,
- include tests for at least three valid expressions,
- include a `build.sh` script that builds the app and tests,
- compile with `-Wall -Wextra -pedantic`.

Suggested work order:

1. Create the project directory structure.
2. Add `build.sh` that compiles an empty program.

3. Define `Token`.
4. Implement tokenization for numbers and operators.
5. Convert tokens to Reverse Polish Notation.
6. Evaluate the RPN expression with a stack.
7. Add tests for valid expressions.
8. Add error handling and tests for invalid expressions.

Extra tasks:

- negative numbers,
- exponentiation,
- constants `pi` and `e`,
- repeated calculations in a loop,
- calculation history,
- comparison with a recursive parser.

11.11 Practice Tasks

1. Split a one-file calculator into `calculator.h`, `calculator.cpp`, and `main.cpp`.
2. Add a `build.sh` script for the calculator.
3. Add a test program that checks three functions.
4. Add a syntax-only command for one starter file.
5. Add a `README.md` section that explains how to build and test the project.
6. Start the RPN calculator with only tokenization and tests.

11.12 Checklist

Before moving on, make sure that your project:

- has headers and source files in clear directories,
- keeps generated files in `build/`,
- has a repeatable build command or script,
- compiles with warnings enabled,
- has tests that return non-zero on failure,
- separates parsing, calculation, and user interaction,
- documents build and test commands.

Chapter 12

09 - Modern C++

This chapter collects selected features from modern C++, mainly C++11 and newer. The goal is not to cover every language feature. The goal is to recognize useful tools that make beginner projects safer and easier to read.

12.1 Learning Goals

After this chapter you should be able to:

- recognize selected modern C++ features,
- use `auto`, `nullptr`, brace initialization, and range-based loops,
- use `enum class`, type aliases, and `override`,
- write simple lambda expressions,
- understand the basic idea of move semantics,
- organize names with namespaces,
- explain the difference between classic file splitting and C++20 modules,
- run a simple `std::thread` example.

12.2 `auto`

`auto` lets the compiler deduce the type from the initializer:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers = {1, 2, 3};

    for (auto value : numbers) {
        std::cout << value << std::endl;
    }
}
```

Use `auto` when the type is obvious from the right side or when the exact type is long, for example an iterator. Do not use it to hide important information.

12.3 `nullptr`

Use `nullptr` instead of 0 or NULL for empty pointers:

```
int* pointer = nullptr;

if (pointer == nullptr) {
```

```

    // no object is available
}

```

`nullptr` has a clear pointer meaning and avoids old C-style ambiguities.

12.4 Brace Initialization

Brace initialization gives a consistent way to initialize values:

```

int count{10};
double price{19.99};
std::vector<int> numbers{1, 2, 3};

```

It is especially useful when initializing containers and objects.

12.5 Range-Based for

Use a range-based loop when you want to process every element:

```

for (const auto& name : names) {
    std::cout << name << std::endl;
}

```

Use `const auto&` when you only read larger objects and want to avoid copying.

12.6 enum class

`enum class` creates a scoped enumeration:

```

enum class Status {
    New,
    InProgress,
    Done
};

```

```

Status status = Status::New;

```

This is safer than an old unscoped `enum` because names do not leak into the surrounding scope.

12.7 Type Aliases

Use `using` to give a meaningful name to a type:

```

using StudentId = int;
using Points = int;

```

Aliases do not create new types, but they can make interfaces easier to read.

12.8 override

Use `override` when a derived class overrides a virtual method:

```

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

class Square : public Shape {
public:

```

```

    double area() const override {
        return 16.0;
    }
};

```

If the method signature is wrong, the compiler will report an error. This is a useful safety net.

12.9 Lambdas

A lambda is a small function object written inline:

```

#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers = {5, 1, 9, 2};

    std::sort(numbers.begin(), numbers.end(),
        [](int a, int b) {
            return a > b;
        });

    for (int value : numbers) {
        std::cout << value << std::endl;
    }
}

```

Lambdas are often used with STL algorithms.

12.10 Basic Move Semantics Idea

Some objects own resources, for example memory inside a `std::vector` or `std::string`. Moving can transfer resources instead of copying them:

```

#include <string>
#include <utility>
#include <vector>

int main() {
    std::vector<std::string> source = {"alpha", "beta"};
    std::vector<std::string> target = std::move(source);
}

```

After moving, the source object is still valid, but its value should not be assumed. For beginner projects, the most important rule is simple: do not use `std::move` unless you know why ownership should be transferred.

12.11 Namespaces

Namespaces group related names and avoid conflicts:

```

namespace math {
    int add(int a, int b) {
        return a + b;
    }
}

int main() {

```

```

    return math::add(2, 3);
}

```

In small exercises, namespaces are optional. In larger projects, they help keep modules organized.

12.12 Classic Files and C++20 Modules

This course mainly uses classic `.h` and `.cpp` files because they are widely supported and easy to build with simple commands.

C++20 modules are a newer language feature that can replace parts of the header model in modern toolchains. Treat modules as a topic for further study after you are comfortable with headers, source files, and build scripts.

12.13 Basic Threads

`std::thread` starts work in a separate thread:

```

#include <iostream>
#include <thread>

void work() {
    std::cout << "Work in another thread" << std::endl;
}

int main() {
    std::thread worker(work);
    worker.join();
}

```

`join` waits for the thread to finish. Multithreading quickly introduces data races and synchronization problems, so keep first examples small.

12.14 Practice Tasks

1. Rewrite a loop over a vector using a range-based `for`.
2. Replace a nullable pointer example with `nullptr`.
3. Create an `enum class` for task status.
4. Sort a vector of records with a lambda.
5. Add `override` to a polymorphic example.
6. Put two utility functions into a namespace.
7. Run a simple `std::thread` example and explain why `join` is needed.

12.15 Checklist

Before moving on, make sure that your code:

- uses modern features to improve clarity, not to look advanced,
- avoids unnecessary `std::move`,
- uses `override` on overridden methods,
- keeps lambdas short and readable,
- still compiles with `-Wall -Wextra -pedantic`,
- stays understandable to another student.

Chapter 13

10 - Projects

This chapter collects larger project assignments. A project should connect the previous course topics: language basics, functions, STL, classes, files, tests, project structure, and short documentation.

13.1 Learning Goals

After this chapter you should be able to:

- choose a project topic and narrow its scope,
- split a solution into smaller modules,
- choose containers and classes for the problem,
- prepare a compilable multi-file project,
- add simple tests or manual verification scenarios,
- write user and technical documentation,
- describe limitations and possible extensions.

13.2 Project Categories

Choose the smallest project that still lets you practice the intended material. A stable minimum version is more valuable than an unfinished large idea.

13.2.1 Console Tools

- logger,
- calculator,
- matrix calculator,
- CSV parser,
- tic-tac-toe.

These are good first projects because they can run in a terminal and usually do not require external libraries.

13.2.2 User-Facing Systems

- order queue,
- vehicle database,
- task management system,
- ATM simulator.

These projects practice classes, containers, validation, file storage, and menu flows.

13.2.3 Games

- Tetris,

- Snake,
- Pac-Man.

Games are larger because they combine state, loops, input, rendering, and many edge cases. They should be treated as extended projects.

13.3 How to Choose a Project

Before accepting a topic, answer these questions:

- Can you describe the minimum version in five bullet points?
- Can the first version run in the terminal without extra libraries?
- Can you build a working version in one or two class sessions?
- Do you know what data the program stores or processes?
- Can you name three verification scenarios?

If several answers are unclear, reduce the scope or choose a simpler topic.

13.4 Minimum Version First

Every project should have a minimum version and optional extensions.

The minimum version:

- compiles,
- runs,
- implements the required core behavior,
- handles at least one invalid input case,
- can be demonstrated from start to finish.

The extended version:

- includes the minimum version,
- adds at least two useful extensions,
- does not break the core behavior,
- documents what was added.

Extensions should never hide a broken minimum version.

13.5 Suggested Project Structure

A practical student project can use this structure:

```
project-name/
  include/
  src/
  tests/
  data/
  build/
  README.md
  build.sh
```

Keep generated files in `build/`. Commit source files, tests, documentation, and small sample input files when they are needed.

13.6 README Checklist

The project `README.md` should include:

- project goal,
- build command,

- run command,
- minimum feature list,
- optional extensions,
- directory structure or important files,
- example input or usage,
- known limitations.

The README does not need to be long. It must be enough for another person to build, run, and understand the project.

13.7 Final Project Checklist

Use this checklist before submitting a project.

Minimum behavior:

- all minimum requirements are implemented,
- the main menu or main usage flow is understandable,
- the basic use case works from start to finish,
- at least one invalid input case is handled,
- the program does not crash on a common user mistake.

Build and run:

- README.md contains a build command,
- README.md contains a run command,
- the project compiles with `-Wall -Wextra -pedantic`,
- the project does not require manually generated files,
- build artifacts are not committed.

Code organization:

- code is split into functions or methods by responsibility,
- `main` starts the program but does not contain all logic,
- source files have clear names,
- larger projects separate declarations and definitions,
- names of variables, functions, classes, and files describe their role.

Data and errors:

- user input is validated,
- file operations check read and write errors,
- an empty data set is handled predictably,
- boundary cases are described or tested,
- error messages help the user correct the input.

Tests and verification:

- the project has automated tests or manual verification scenarios,
- there is one valid-data scenario,
- there is one invalid-data scenario,
- there is one boundary scenario, such as an empty list, zero, or missing file,
- the student can repeat the scenarios during review.

Documentation and demo:

- README.md describes the project goal,
- README.md separates the minimum version from extensions,
- limitations or known gaps are described honestly,
- the student can point to the most important functions, classes, or modules,
- the student can explain what could be added in the next version.

13.8 Grading Rubric

Use this simple 100-point rubric:

Area	Points	What is evaluated
Minimum functionality	30	All minimum requirements work.
Technical correctness	20	The project compiles with warnings enabled, avoids committed build artifacts, and handles invalid input.
Code organization	20	Code is split into functions, classes, and files that match the problem. Names are clear.
Tests or verification scenarios	15	The author can show valid, invalid, and boundary cases.
Documentation	10	README.md explains build, run, structure, limitations, and usage.
Extensions	5	Useful additions are implemented without breaking the minimum version.

Review should start with the minimum version. If the minimum version does not compile or run, extensions should not compensate for it.

13.9 Technical Review Questions

During project review, be ready to explain:

- where the program reads data,
- where the program writes data,
- where the most important logic is located,
- how user errors are handled,
- why you chose specific containers or classes,
- what you would change in the next version.

13.10 Suggested Project Topics

Logger:

- writes messages to a text file,
- supports levels such as info, warning, and error,
- practices files, validation, and timestamps.

Calculator:

- evaluates arithmetic expressions,
- can start with simple operations,
- can later add history, parentheses, or functions.

CSV parser:

- reads rows from a text file,
- validates column counts,
- reports malformed input.

Vehicle database:

- stores vehicles in a collection,
- supports add, list, search, and save,
- practices classes and file storage.

Task manager:

- stores tasks with status and priority,
- supports add, complete, filter, and save,
- practices STL, classes, and validation.

ATM simulator:

- models account balance and operations,
- validates withdrawals and deposits,
- practices state, menu flow, and tests.

Games:

- start with console or simple rendering,
- keep the model of the game separate from drawing,
- define clear winning, losing, and restart rules.

13.11 Final Advice

Build projects in small steps:

1. Create the directory structure.
2. Add a minimal README with build and run commands.
3. Write the smallest program that compiles.
4. Add the data model or first class.
5. Implement one minimum feature.
6. Run it and write one verification scenario.
7. Add the next feature only after the previous one works.

This approach makes progress visible and keeps the project review concrete.